Building a Ready-to-Use TUI Coding Agent: Harness, Tools, Skills, and Project Memory

Siyuan Liang | Joe Zou | Ni Li

CS 5100 - Foundations of Artificial Intelligence | Project Proposal | Fall 2026

1. Introduction

We propose to build a lightweight, terminal-based coding agent inspired by OpenCode. Our goal is a ready-to-use text user interface (TUI): after installation and model API configuration, a developer can open a repository, describe a coding task, inspect the agent's actions and changes, and review the result without manually assembling an agent framework. The initial target is small Python repositories and tasks such as bug repair, bounded feature implementation, and test-driven refinement. We will use an existing language model rather than train a new foundation model.

The central focus is the coding harness: the software surrounding the model that manages tool execution, observations, context, state, and stopping conditions. Generating plausible code is insufficient if an agent cannot locate relevant files, recover from failed edits, or verify its changes. We aim to understand how these mechanisms work by implementing a compact system and measuring its behavior. The project connects AI agent design with practical developer tools and could support accessible, inspectable automation for software maintenance and programming education.

Our research question is: under a fixed model and execution budget, how do reusable task instructions (skills) and persistent project memory affect the correctness and efficiency of a tool-using coding agent? We will distinguish product usability from coding performance: a convenient TUI does not itself demonstrate improved task success.

2. Previous Work and Proposed Contributions

SWE-agent [1] studies agent-computer interfaces for repository navigation, code editing, and execution. Its main contribution is showing that the interface supplied to a model matters for software-engineering performance. We value its concrete treatment of tool feedback and interaction design. However, its software-repair setting does not by itself establish the benefits of persistent project memory or instruction-based skills across task types. It motivates our file-search, edit, and test-feedback loop.

The OpenHands Software Agent SDK [2] presents a modular architecture for software agents, including tools, event-based state management, context handling, and local or remote execution. We value its separation of agent logic from execution infrastructure and its support for extensibility, including MCP. Its production-oriented scope is broader than our course project; adopting the complete system would also make individual design choices harder for us to isolate. We will implement a smaller harness with explicit interfaces and execution logs.

OPENDEV [3] describes a terminal-native coding agent with planning/execution separation, lazy tool discovery, context compaction, and cross-session project memory. It is particularly relevant to our intended user experience and context-management goals. We value its detailed engineering account, but the cited version is a work-in-progress preprint. Its combined architecture is not evidence that every component will help our tasks. We will test selected mechanisms separately instead of assuming the entire design improves performance.

3. Methodology and System Design

Our contribution will be an independently implemented, inspectable prototype and a controlled evaluation of selected harness components, rather than a claim to invent coding agents, MCP, or skills. OpenCode is the product inspiration; the three papers inform specific architecture choices. Any reused open-source implementation will be attributed and its license respected. We have not yet written implementation code or collected experimental results.

3.1 Core harness and terminal experience

We plan to use Python with a TUI library such as Textual. The interface will show conversation history, current tool activity, test output, and a proposed diff. A small provider adapter will connect to a tool-capable language-model API. The execution loop will accept a task, request a model action, validate the tool arguments, execute that action, append the observation, and continue until completion, a configured limit, or a user interruption. Task state and a chronological action log will support inspection and session recovery.

The initial tool set will include repository listing, content search, bounded file reads, patch-based editing, and test execution. Tool responses will contain structured status, relevant output, and explicit errors. File edits will be restricted to the selected workspace. Commands will have timeouts and output limits; destructive or out-of-workspace operations will require confirmation. Evaluation runs will use disposable, isolated repository copies. Passing visible tests will be reported as verification evidence, not as proof that every requirement is satisfied.

3.2 MCP and reusable skills

We will implement an MCP client adapter and demonstrate one server-backed tool alongside native tools. Both will use the same internal registry and argument validation, so the agent loop need not depend on how a tool is hosted. MCP provides interoperability; we will not assume that using the protocol alone improves coding accuracy. Compared with directly hard-coded integrations, this design requires extra protocol and failure handling but makes tool replacement easier.

Skills will be versioned instruction files describing procedures such as investigating a failing test or validating a patch. The harness will expose short skill descriptions and load the selected instructions on demand. We prefer this approach to putting every instruction in the initial prompt, which is simpler but can waste context and introduce irrelevant guidance. Skills will provide procedures rather than solutions to evaluation tasks. We will record which skills were loaded and whether their recommended checks were actually executed.

3.3 Memory, context, and optional agent coordination

We will separate temporary conversation state from persistent project memory. Persistent entries will record facts such as test commands, repository structure, and project conventions, with their source and update time. Recent observations and a concise progress summary will form the active context; long logs will remain on disk for retrieval. A local structured store is the initial choice because it is easy to inspect. Vector retrieval is an alternative for larger projects, but adds indexing and relevance errors that are unnecessary for the first prototype.

A single agent will be the minimum deliverable. A separate planning or review agent is a stretch goal: it may provide a useful second check, but increases cost and coordination complexity. Implementation will proceed through four milestones: working TUI and tool loop; MCP integration and skills; memory and recovery; evaluation and packaging. We will prioritize a reliable end-to-end coding workflow over replicating every OpenCode feature.

4. Expected Results and Evaluation Plan

We will prepare 12 bounded tasks across small Python repositories: four bug repairs, four feature additions, and four changes requiring repository conventions or cross-file understanding. Each task will have an initial repository snapshot, a specification, visible tests, and separate held-out acceptance tests. We will freeze tasks, tests, and skill definitions before final evaluation and use separate development tasks to tune prompts. The agent will not be able to inspect or modify the held-out tests.

We will compare four configurations: (A) the core tool-using harness; (B) A plus skills; (C) A plus persistent project memory; and (D) A plus both. The model version, sampling settings, native/MCP tool availability, context limit, and per-task step and token budgets will be held constant. Each configuration will run twice per task, giving 96 planned runs. We will pilot the cost before the full experiment and report any reduction in scope transparently. A one-shot code-generation baseline may be added if resources permit.

For memory-enabled conditions, all configurations will receive the same preliminary repository-orientation opportunity. Only the memory-enabled versions may carry its documented project facts into the evaluated task; they will not retain solutions, patches, or acceptance-test details from evaluation attempts. Every trial will start from a clean repository snapshot. We will separate the cost of preparing memory from the cost of solving the task and report both, rather than treating memory preparation as free.

The primary metric will be task success: all held-out acceptance tests pass without regressions in the existing test suite. Secondary metrics will include total input/output tokens, wall-clock time, tool calls, invalid tool calls, and failed test/repair cycles. We will retain trajectories and diffs, report per-task outcomes and aggregate rates, and inspect failures such as incorrect localization, repeated unproductive actions, stale memory, and premature completion. The small task set and two repeats limit generalization; we will avoid claiming broad superiority over established agents.

We expect the basic harness to work best on localized bugs with clear test feedback. Skills may improve procedural consistency, while memory may reduce repeated discovery of repository conventions. Benefits may be smaller on simple tasks, and irrelevant skills or outdated memory may increase errors or token use. Cross-file changes and underspecified requests are expected to remain harder. These are hypotheses, not measured results. If a mechanism hurts performance, we will analyze its retrieval and context effects rather than report only successful examples.

The intended deliverables are an installable TUI application with setup instructions, a reproducible evaluation runner and task suite, and a report connecting measured outcomes to harness design. Usability checks will cover clean installation, model configuration, opening a repository, completing a small coding task, reviewing a diff, and restoring a saved session. These functional checks will be reported separately from the AI performance evaluation.

References

[1] John Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. arXiv:2405.15793. https://arxiv.org/abs/2405.15793

[2] Xingyao Wang et al. The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents. MLSys 2026; arXiv:2511.03690, v2. https://arxiv.org/abs/2511.03690

[3] Nghi D. Q. Bui. Building Effective AI Coding Agents for the Terminal: Scaffolding, Harness, Context Engineering, and Lessons Learned. 2026. arXiv:2603.05344, v3. Work-in-progress preprint. https://arxiv.org/abs/2603.05344